



4.5.1 Rank-Revealing QR (RRQR) via MGS ¶

The discussion in [Subsection 4.4.4](#) falls short of being a practical algorithm for at least two reasons:

- One needs to be able to determine in advance what columns of A are linearly independent; and
- Due to roundoff error or error in the data from which the matrix was created, a column may be linearly independent of other columns when for practical purposes it should be considered dependent.

We now discuss how the MGS algorithm can be modified so that appropriate linearly independent columns can be determined "on the fly" as well as the defacto rank of the matrix. The result is known as the **Rank Revealing QR factorization (RRQR)**. It is also known as **QR factorization with column pivoting**. We are going to give a modification of the MGS algorithm for computing the RRQR.

For our discussion, we introduce an elementary pivot matrix, $\tilde{P}(j) \in \mathbb{C}^{n \times n}$, that swaps the first element of the vector to which it is applied with the element indexed with j :

$$\tilde{P}(j)x = \begin{pmatrix} e_j^T x \\ e_1^T x \\ \vdots \\ e_{j-1}^T x \\ e_0^T x \\ e_{j+1}^T x \\ \vdots \\ e_{n-1}^T x \end{pmatrix} x = \begin{pmatrix} e_j^T x \\ e_1^T x \\ \vdots \\ e_{j-1}^T x \\ e_0^T x \\ e_{j+1}^T x \\ \vdots \\ e_{n-1}^T x \end{pmatrix} = \begin{pmatrix} \chi_j \\ \chi_1 \\ \vdots \\ \chi_{j-1} \\ \chi_0 \\ \chi_{j+1} \\ \vdots \\ \chi_{n-1} \end{pmatrix}.$$

Another way of stating this is that

$$\tilde{P}(j) = \begin{pmatrix} 0 & 0 & 1 & 0 \\ 0 & I_{(j-1) \times (j-1)} & 0 & 0 \\ 1 & 0 & 0 & 0 \\ 0 & 0 & 0 & I_{(n-j-1) \times (n-j-1)} \end{pmatrix},$$

where $I_{k \times k}$ equals the $k \times k$ identity matrix. When applying $\tilde{P}(j)$ from the right to a matrix, it swaps the first column and the column indexed with j . Notice that $\tilde{P}(j)^T = \tilde{P}(j)$ and $\tilde{P}(j) = \tilde{P}(j)^{-1}$.

Remark 4.5.1.1. For a more detailed discussion of permutation matrices, you may want to consult Week 7 of "Linear Algebra: Foundations to Frontiers" (LAFF) [\[27\]](#). We also revisit this in [Section 5.3](#) when discussing LU factorization with partial pivoting.

Here is an outline of the algorithm:

- Determine the index π_1 such that the column of A indexed with π_1 has the largest 2-norm (is the longest).
- Permute $A := A\tilde{P}(\pi_1)$, swapping the first column with the column that is longest.
- Partition

$$A \rightarrow \begin{pmatrix} a_1 & A_2 \end{pmatrix}, Q \rightarrow \begin{pmatrix} q_1 & Q_2 \end{pmatrix}, R \rightarrow \begin{pmatrix} \rho_{11} & r_{12}^T \\ 0 & R_{22} \end{pmatrix}, p \rightarrow \begin{pmatrix} \pi_1 \\ p_2 \end{pmatrix}$$

- Compute $\rho_{11} := \|a_1\|_2$.
- $q_1 := a_1 / \rho_{11}$.
- Compute $r_{12}^T := q_1^T A_2$.
- Update $A_2 := A_2 - q_1 r_{12}^T$.

This subtracts the component of each column that is in the direction of q_1 .

- Continue the process with the updated matrix A_2 .

The complete algorithm, which overwrites A with Q , is given in [Figure 4.5.1.2](#). Observe that the elements on the diagonal of R will be positive and in non-increasing order because updating $A_2 := A_2 - q_1 r_{12}^T$ inherently does not increase the length of the columns of A_2 . After all, the component in the direction of q_1 is being subtracted from each column of A_2 , leaving the component orthogonal to q_1 .

```

[ $A, R, p$ ] := RRQR_MGS_simple( $A, R, p$ )

 $A \rightarrow \left( \begin{array}{c|c} A_L & A_R \end{array} \right), R \rightarrow \left( \begin{array}{c|c} R_{TL} & R_{TR} \\ \hline R_{BL} & R_{BR} \end{array} \right), p \rightarrow \left( \begin{array}{c} p_T \\ \hline p_B \end{array} \right)$ 

 $A_L$  has 0 columns,  $R_{TL}$  is  $0 \times 0$ ,  $p_T$  has 0 rows

while  $n(A_L) < n(A)$ 
     $\left( \begin{array}{c|c} A_L & A_R \end{array} \right) \rightarrow \left( \begin{array}{c|cc} A_0 & a_1 & A_2 \end{array} \right),$ 
     $\left( \begin{array}{c|c} R_{TL} & R_{TR} \\ \hline R_{BL} & R_{BR} \end{array} \right) \rightarrow \left( \begin{array}{ccc|c} R_{00} & r_{01} & R_{02} & \\ \hline r_{10}^T & \rho_{11} & r_{12}^T & \\ R_{20} & r_{21} & R_{22} & \end{array} \right), \left( \begin{array}{c} p_T \\ \hline p_B \end{array} \right) \rightarrow \left( \begin{array}{c} p_0 \\ \hline \pi_1 \\ p_2 \end{array} \right)$ 
     $\pi_1 = \text{DetermineColumnIndex}((a_1 \quad A_2))$ 
     $(a_1 \quad A_2) := (a_1 \quad A_2) \tilde{P}(\pi_1)$ 
     $\rho_{11} := \|a_1\|_2$ 
     $a_1 := a_1 / \rho_{11}$ 
     $r_{12}^T := a_1^T A_2$ 
     $A_2 := A_2 - a_1 r_{12}^T$ 
     $\left( \begin{array}{c|c} A_L & A_R \end{array} \right) \leftarrow \left( \begin{array}{c|cc} A_0 & a_1 & A_2 \end{array} \right),$ 
     $\left( \begin{array}{c|c} R_{TL} & R_{TR} \\ \hline R_{BL} & R_{BR} \end{array} \right) \leftarrow \left( \begin{array}{ccc|c} R_{00} & r_{01} & R_{02} & \\ \hline r_{10}^T & \rho_{11} & r_{12}^T & \\ R_{20} & r_{21} & R_{22} & \end{array} \right), \left( \begin{array}{c} p_T \\ \hline p_B \end{array} \right) \leftarrow \left( \begin{array}{c} p_0 \\ \hline \pi_1 \\ p_2 \end{array} \right)$ 
endwhile

```

Figure 4.5.1.2. Simple implementation of RRQR via MGS. Incorporating a stopping criteria that checks whether ρ_{11} is small would allow the algorithm to determine the effective rank of the input matrix.

The problem with the algorithm in [Figure 4.5.1.2](#) is that determining the index π_1 requires the 2-norm of all columns in A_R to be computed, which costs $O(m(n-j))$ flops when A_L has j columns (and hence A_R has $n-j$ columns).

The following insight reduces this cost: Let $A = \left(\begin{array}{c|c} a_0 & a_1 \mid \cdots \mid a_{n-1} \end{array} \right)$,

$$v = \begin{pmatrix} \nu_0 \\ \nu_1 \\ \vdots \\ \nu_{n-1} \end{pmatrix} = \begin{pmatrix} \|a_0\|_2^2 \\ \|a_1\|_2^2 \\ \vdots \\ \|a_{n-1}\|_2^2 \end{pmatrix}, q^T q = 1 \text{ (here } q \text{ is of the same size as the columns}$$

$$\text{of } A), \text{ and } r = A^T q = \begin{pmatrix} \rho_0 \\ \rho_1 \\ \vdots \\ \rho_{n-1} \end{pmatrix}. \text{ Compute } B := A - qr^T \text{ with}$$

$$B = \left(\begin{array}{c|c} b_0 & b_1 \mid \cdots \mid b_{n-1} \end{array} \right). \text{ Then}$$

$$\begin{pmatrix} \|b_0\|_2^2 \\ \|b_1\|_2^2 \\ \vdots \\ \|b_{n-1}\|_2^2 \end{pmatrix} = \begin{pmatrix} \nu_0 - \rho_0^2 \\ \nu_1 - \rho_1^2 \\ \vdots \\ \nu_{n-1} - \rho_{n-1}^2 \end{pmatrix}.$$

To verify this, notice that

$$a_i = (a_i - a_i^T q q) + a_i^T q q$$

and

$$(a_i - a_i^T q q)^T q = a_i^T q - a_i^T q q^T q = a_i^T q - a_i^T q = 0.$$

This means that

$$\|a_i\|_2^2 = \|(a_i - a_i^T q q) + a_i^T q q\|_2^2 = \|a_i - a_i^T q q\|_2^2 + \|a_i^T q q\|_2^2 = \|a_i - \rho_i q\|_2^2 + \|\rho_i q\|_2^2 = \|b_i\|_2^2 + \rho_i^2$$

so that

$$\|b_i\|_2^2 = \|a_i\|_2^2 - \rho_i^2 = \nu_i - \rho_i^2.$$

Building on this insight, we make an important observation that greatly reduces the cost of determining the column that is longest. Let us start by computing v as the vector such that the i th entry in v equals the square of the length of the i th column of A . In other words, the i th entry of v equals the dot product of the i th column of A with itself. In the above outline for the MGS with column pivoting, we can then also partition

$$v \rightarrow \begin{pmatrix} \nu_1 \\ v_2 \end{pmatrix}.$$

The question becomes how v_2 before the update $A_2 := A_2 - q_1 r_{12}^T$ compares to v_2 after that update. The answer is that the i th entry of v_2 must be updated by subtracting off the square of the i th entry of r_{12}^T .

Let us introduce the functions $v = \text{ComputeWeights}(A)$ and $v = \text{UpdateWeights}(v, r)$ to compute the described weight vector v and to update a weight vector v by subtracting from its elements the squares of the corresponding entries of r . Also, the function DeterminePivot returns the index of the largest in the vector, and swaps that entry with the first entry. An optimized RRQR via MGS algorithm, RRQR-MGS, is now given in [Figure 4.5.1.3](#). In that algorithm, A is overwritten with Q .

$[A, R, p] := \text{RRQR_MSG}(A, R, p)$
$v := \text{ComputeWeights}(A)$
$A \rightarrow \left(\begin{array}{c c} A_L & A_R \end{array} \right), R \rightarrow \left(\begin{array}{c c} R_{TL} & R_{TR} \\ \hline R_{BL} & R_{BR} \end{array} \right), p \rightarrow \left(\begin{array}{c} p_T \\ \hline p_B \end{array} \right), v \rightarrow \left(\begin{array}{c} v_T \\ \hline v_B \end{array} \right)$
A_L has 0 columns, R_{TL} is 0×0 , p_T has 0 rows, v_T has 0 rows
while $n(A_L) < n(A)$
$\left(\begin{array}{c c} A_L & A_R \end{array} \right) \rightarrow \left(\begin{array}{c cc} A_0 & a_1 & A_2 \end{array} \right),$
$\left(\begin{array}{c c} R_{TL} & R_{TR} \\ \hline R_{BL} & R_{BR} \end{array} \right) \rightarrow \left(\begin{array}{c cc} R_{00} & r_{01} & R_{02} \\ \hline r_{10}^T & \rho_{11} & r_{12}^T \\ \hline R_{20} & r_{21} & R_{22} \end{array} \right),$
$\left(\begin{array}{c} p_T \\ \hline p_B \end{array} \right) \rightarrow \left(\begin{array}{c} p_0 \\ \hline \pi_1 \\ \hline p_2 \end{array} \right), \left(\begin{array}{c} v_T \\ \hline v_B \end{array} \right) \rightarrow \left(\begin{array}{c} v_0 \\ \hline \nu_1 \\ \hline v_2 \end{array} \right)$
$\left[\left(\begin{array}{c} \nu_1 \\ \hline v_2 \end{array} \right), \pi_1 \right] = \text{DeterminePivot} \left(\left(\begin{array}{c} \nu_1 \\ \hline v_2 \end{array} \right) \right)$
$\left(\begin{array}{c cc} A_0 & a_1 & A_2 \end{array} \right) := \left(\begin{array}{c cc} A_0 & a_1 & A_2 \end{array} \right) \left(\begin{array}{c c} I & 0 \\ \hline 0 \bar{P}(\pi_1)^T & \end{array} \right)$
$\rho_{11} := \ a_1\ _2$
$a_1 := a_1 / \rho_{11}$
$r_{12}^T := q_1^T A_2$
$A_2 := A_2 - q_1 r_{12}^T$
$v_2 := \text{UpdateWeights}(v_2, r_{12})$
$\left(\begin{array}{c c} A_L & A_R \end{array} \right) \leftarrow \left(\begin{array}{c cc} A_0 & a_1 & A_2 \end{array} \right),$
$\left(\begin{array}{c c} R_{TL} & R_{TR} \\ \hline R_{BL} & R_{BR} \end{array} \right) \leftarrow \left(\begin{array}{c cc} R_{00} & r_{01} & R_{02} \\ \hline r_{10}^T & \rho_{11} & r_{12}^T \\ \hline R_{20} & r_{21} & R_{22} \end{array} \right),$
$\left(\begin{array}{c} p_T \\ \hline p_B \end{array} \right) \leftarrow \left(\begin{array}{c} p_0 \\ \hline \pi_1 \\ \hline p_2 \end{array} \right), \left(\begin{array}{c} v_T \\ \hline v_B \end{array} \right) \leftarrow \left(\begin{array}{c} v_0 \\ \hline \nu_1 \\ \hline v_2 \end{array} \right)$
endwhile

Figure 4.5.1.3. RRQR via MGS, with optimization. Incorporating a stopping criteria that checks whether ρ_{11} is small would allow the algorithm to determine the effective rank of the input matrix.

Let us revisit the fact that the diagonal elements of R are positive and in nonincreasing order. This upper triangular matrix is singular if a diagonal element equals zero (and hence all subsequent diagonal elements equal zero). Hence, if ρ_{11} becomes small relative to prior diagonal elements, the remaining columns of the (updated) A_R are essentially zero vectors, and the original matrix can be approximated with

$$A \approx Q_L \begin{pmatrix} R_{TL} & R_{TR} \end{pmatrix} = \begin{bmatrix} \\ \\ \\ \end{bmatrix} \begin{bmatrix} & & & \end{bmatrix}$$

If Q_L has k columns, then this becomes a rank- k approximation.

Remark 4.5.1.4. Notice that in updating the weight vector v , the accuracy of the entries may progressively deteriorate due to catastrophic cancellation. Since these values are only used to determine the order of the columns and, importantly, when they become very small the rank of the matrix has revealed itself, this is in practice not a problem.